

Django

Pages App – Chapter 3

Material in these slides was taken from **DJANGO for Beginners** – William S. Vincent

Initial Set Up

Shell

Windows

\$ cd onedrive\desktop\code

\$ mkdir personal_website

\$ cd personal_website

\$ python -m venv .venv

\$.venv\Scripts\Activate.ps1

(.venv) \$

macOS

\$ cd ~/desktop/code

\$ mkdir personal_website

\$ cd personal_website

\$ python3 -m venv .venv

\$ source .venv/bin/activate

(.venv) \$

Initial Set Up

Shell

```
(.venv) $ python -m pip install django~=5.0.0 black  
(.venv) $ django-admin startproject django_project .  
(.venv) $ python manage.py startapp pages
```

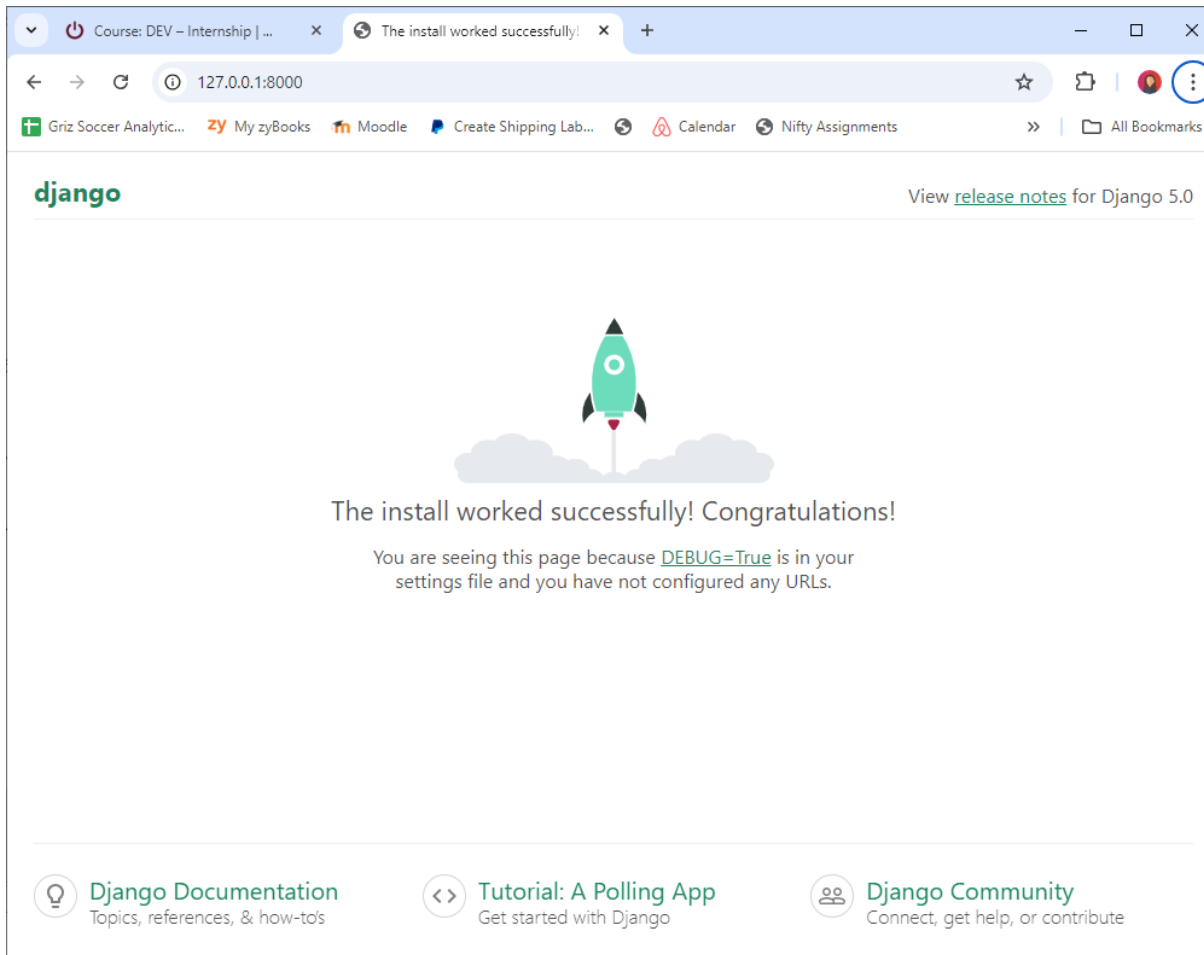
Code

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "pages", # new
]
```

Initialize the database with `migrate` and start the local web server with `runserver`.

Shell

```
(.venv) $ python manage.py migrate  
(.venv) $ python manage.py runserver
```



A Django “view” is a Python function that accepts a Web request and returns a Web response. When a Web page is requested, Django automatically creates an `HttpRequest` object with metadata about the request. The view returns an `HttpResponse` object.

We will use `render` for the About page, so leaving that import as is is fine. For our home page, we will repeat the steps from the previous chapter: import the class `HttpResponse`, create a view named `home_page_view`, name the first parameter (which is always the `HttpRequest` object!) as `request`, and then return the text “Homepage.”

Code

```
# pages/views.py
from django.http import HttpResponse
from django.shortcuts import render

def home_page_view(request):
    return HttpResponse("Homepage")
```

The next step is creating a `urls.py` file within the `pages` app that imports `path` from `django.urls`, `home_page_view` from the `views.py` file in the local directory, and sets a route at the empty string, `""`, that calls `home_page_view`.

Code

```
# pages/urls.py
from django.urls import path

from .views import home_page_view

urlpatterns = [
    path("", home_page_view),
]
```

The final step is updating the project-level `urls.py` file, the initial entry point for all URL requests. We import the `include` function to include other URL configurations and set a URL route of `""` for the `pages` application.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include  # new

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("pages.urls")),  # new
]
```

That's it! If the local server is still running on the command line, you should be able to visit the home page in your browser:



Homepage

Function-Based View About Page

`render()`³⁹ is a Django shortcut function useful for working with a template. The first parameter in `render()` is the request object, and the second parameter is a template name. That means if we want to create a view for an about page using a template called `about.html` we can write it like this:

Code

```
# pages/views.py
from django.http import HttpResponse
from django.shortcuts import render

def home_page_view(request):
    return HttpResponse("Homepage")

def about_page_view(request): # new
    return render(request, "pages/about.html")
```

Templates

Templates are the presentation layer in Django, providing a convenient way to generate HTML files that can *also* include CSS, JavaScript, and media such as images. By default, Django's template loading engine looks for a “templates” subdirectory in each app. That means we *could* add a `templates` directory within the `pages` app and then include the `about.html` in the following way:

Layout

```
└─ pages
   └─ templates
      └─ about.html
```

This approach works, but it is not a best practice. Django chooses the first template it finds whose name matches. What happens if there are `about.html` files within two separate apps? Django cannot easily tell which one should be used in our function. Fortunately, a simple fix is to *namespace* template files by placing them within another directory containing the application name. In other words: `app -> templates -> app -> template file`.

Then, add a new file called `about.html` within the `pages/templates/pages` directory. You can do this in Visual Studio Code by navigating with your mouse to the top left of your screen, clicking “File,” and then “New File.” Make sure to name and save the file in the correct location.

The `about.html` file will have a `<h1>` tag for a headline and a `<p>` tag for paragraph text.

Code

```
<!-- pages/templates/pages/about.html -->
<h1>About page</h1>
<p>This is the new template-powered About page.</p>
```

URL Dispatcher

At this point, the pattern for adding a new URL route should start to feel familiar. We import our view, `about_page_view`, and then set its path, `about/`, and the view name.

Code

```
# pages/urls.py
from django.urls import path

from .views import home_page_view, about_page_view # new

urlpatterns = [
    path("about/", about_page_view), # new
    path("", home_page_view),
]
```

If you navigate to `http://127.0.0.1:8000/about/`, the About page will be visible.



About page

This is the new template-powered About page.

FBV About Page

The Django Template Language

All web frameworks (including Django) need a way to generate HTML dynamically, and the most common approach is to use templates that contain static content and a special syntax for inserting dynamic content. Django has its own [templating language](#)⁴⁰ that is not attempting to replace dedicated JavaScript front-ends like Vue, React, or Angular. Rather, it is a deliberately limited—but still powerful—way to use comments, variables, filters, tags, and template inheritance.

Let's revisit our `about.html` template to try out some of these features. We'll begin with `comment`⁴¹ tags that allow us to add code that is readable to the developer but not displayed on the page. Anything between `{% comment %}` and `{% endcomment %}` tags will be ignored.

Code

```
<!-- pages/templates/pages/about.html -->
<h1>About page</h1>
<p>This is the new template-powered About page.</p>
{% comment %}This is just a comment. It won't appear on the web page!{% endcomment %}
```

Template Context

Whenever a Django template is rendered, a `context`⁴³ is also created. This context is a dictionary-like object with variable names for keys and variable values as values. We can and will update this context with information, often from a database. When you render a template with a given context, every key in the context dictionary becomes a variable in the template that you can access and use.

The `render()`⁴⁴ shortcut function expects parameters in the following order:

1. The `HttpRequest` object conventionally named `request`
2. The `template_name`
3. The context dictionary

In other words: `render(request, template_name, context)`. By default, context is set to `None`, but we can manually add information to see it in action.

Code

```
# pages/views.py
from django.http import HttpResponseRedirect
from django.shortcuts import render

def home_page_view(request):
    return HttpResponseRedirect("Homepage")

def about_page_view(request):
    context = {"name": "Alice"} # new
    return render(request, "pages/about.html", context) # new
```

Here, we've added a variable, `context`, containing a dictionary with the key variable of "name" and a value variable of "Alice." To display the context in our template, the syntax for [variables](#)⁴⁵ is to surround the variable name with brackets `{}`. Since the key value is "name," we type `{{ name }}` to make it visible.

Code

```
<!-- pages/templates/pages/about.html -->  
<h1>About page</h1>  
<p>My name is {{ name }}.</p>
```

If you refresh your browser, it will now show the name "Alice."



About page

My name is Alice.

About Page with Name

Code

```
# pages/views.py
from django.http import HttpResponseRedirect
from django.shortcuts import render

def home_page_view(request):
    return HttpResponseRedirect("Homepage")

def about_page_view(request):
    context = {
        "name": "Alice",
        "age": 33, # new
    }
    return render(request, "pages/about.html", context)
```

Code

```
<!-- pages/templates/pages/about.html -->  
<h1>About page</h1>  
<p>My name is {{ name }}. I am {{ age }} years old.</p>
```

Tests

Writing tests for your code is as important as writing the code itself. Without accompanying tests, you can't submit new code changes to an open-source project or within any well-structured company.

The reasoning is simple: while your code change may be small, there is no telling what else it might inadvertently break in the project.

Writing good tests provides confidence in your codebase, doesn't take too long, and can be automated to run automatically on any new code changes.

Testing can be divided into two main categories: **unit** and **integration**.

Unit tests check a piece of functionality in isolation, while **Integration tests** check multiple linked pieces.

Anytime you create new functionality, **a test is necessary** to confirm that it works as intended. For example, in our project, we have a home page and an about page, and we should test that both exist at the expected URLs. It may seem unnecessary now, but as a project grows in size, there's no telling what can change.

The Python standard library contains a built-in testing framework called **unittest** that uses **TestCase** instances and a long list of **assert methods** to check for and report failures. Django's testing framework provides several extensions on top of Python's **unittest.TestCase** base class.

Code

```
# pages/tests.py
from django.test import SimpleTestCase

class HomepageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

class AboutpageTests(SimpleTestCase):
    def test_url_exists_at_correct_location(self):
        response = self.client.get("/about/")
        self.assertEqual(response.status_code, 200)
```

For our first tests, we'll check that the two URLs for our website, the Homepage and About page, return HTTP 200 status codes, the standard response for a successful HTTP request

To run the tests, quit the server with Control+c and type `python manage.py test` on the command line to run them.

Shell

```
(.venv) $ python manage.py test
Found 2 test(s).
System check identified no issues (0 silenced).
```

```
..
```

```
-----
```

```
Ran 2 tests in 0.003s
```

```
OK
```

Shell

```
(.venv) $ git init  
(.venv) $ git status
```

.gitignore

```
.venv/  
__pycache__/  
db.sqlite3
```

Shell

```
(.venv) $ pip freeze > requirements.txt
```

Shell

```
(.venv) $ git status
```

```
(.venv) $ git add -A
```

```
(.venv) $ git commit -m "initial commit"
```

Shell

```
(.venv) $ git remote add origin https://github.com/wsvincent/personal-website.git  
(.venv) $ git branch -M main  
(.venv) $ git push -u origin main
```
